

Trabajo Práctico: Funciones y Manejo de Eventos en C# WinForms

Objetivo

Diseñar una interfaz de escritorio en **Windows Forms (C#)** que interactúe con el usuario a través de controles de entrada (TextBox, NumericUpDown, ComboBox, RadioButton), capturar eventos de la interfaz y procesar la información invocando **funciones externas a los eventos** para mantener un código limpio y reutilizable.

Ejercicio 1: Calculadora de Descuentos y Categorización

Escenario: El sistema de un comercio necesita calcular el importe final de una venta según el tipo de cliente y aplicar recargos o descuentos mediante funciones.

Diseño del Formulario (FormCalculadora)

- TextBox (txtCliente): Nombre del cliente.
- NumericUpDown (numMonto): Monto base de la compra.
- ComboBox (cmbTipoCliente): Opciones: "Regular", "Socio", "VIP".
- RadioButton (rbEfectivo / rbTarjeta): Medio de pago.
- Button (btnCalcular): Text = "Calcular Total".
- Label (lblResultado): Muestra el desglose final.

Lógica Requerida (Métodos/Funciones)

Crea las siguientes funciones con visibilidad private o dentro de una clase auxiliar:

1. **CalcularPorcentajeDescuento(string tipoCliente):** Devuelve un decimal (0.0 para Regular, 0.10 para Socio, 0.20 para VIP).
2. **CalcularRecargoMedioPago(bool esTarjeta, decimal monto):** Si se paga con tarjeta, retorna un recargo del 5% ($\text{monto} * 0.05$), caso contrario 0.
3. **ObtenerMontoFinal(decimal montoBase, decimal descuento, decimal recargo):** Devuelve $\text{montoBase} - \text{descuento} + \text{recargo}$.

Requisito de Código

En el evento btnCalcular_Click, no debes realizar cálculos matemáticos directamente. El evento debe:

- Leer las entradas del formulario.
- Validar que los campos no estén vacíos.
- Invocarlas 3 funciones creadas.
- Formatear el resultado y asignarlo a lblResultado.Text.

Ejercicio 2: Validador de Fortalezas de Contraseñas

Escenario: Una pantalla de registro de usuarios requiere validar el formato de una clave antes de enviarla a la base de datos.

Diseño del Formulario (FormPassword)

- TextBox (txtPassword): Usar la propiedad UseSystemPasswordChar = true.
- CheckBox (chkMostrarPass): Text = *"Mostrar contraseña"*.
- Label (lblEvaluacion): Cambia de texto y color de fondo según el resultado.

Eventos a Programar

- txtPassword_TextChanged: Se dispara automáticamente con cada tecla que presiona el usuario.
- chkMostrarPass_CheckedChanged: Muestra u oculta los caracteres del TextBox.

Lógica Requerida (Funciones)

1. **EsLongitudValida(string pass):** Devuelve true si tiene al menos 8 caracteres.
2. **TieneNumeroYMayuscula(string pass):** Recorre el string y devuelve true si contiene al menos un dígito (char.IsDigit) y una letra mayúscula (char.IsUpper).
3. **EvaluarNivelSeguridad(string pass):** Retorna un string (*"Débil"*, *"Media"*, *"Fuerte"*) según el cumplimiento de las reglas anteriores.

Requisito en el Evento

En el evento TextChanged, invoca la función EvaluarNivelSeguridad y actualiza la interfaz en tiempo real:

- Si es *"Débil"*: lblEvaluacion.ForeColor = Color.Red.
- Si es *"Media"*: lblEvaluacion.ForeColor = Color.Orange.
- Si es *"Fuerte"*: lblEvaluacion.ForeColor = Color.Green.

Ejercicio 3: Conversor de Unidades con Retorno de Tuplas / Parámetros out

Escenario: Formulario para convertir unidades numéricas de temperatura o longitud.

Diseño del Formulario (FormConversor)

- NumericUpDown (numValor): Valor a convertir.

- RadioButton (rbCelsToFahr / rbFahrToCels): Dirección de la conversión.
- Button (btnConvertir): Ejecuta la conversión.
- Label (lblResumen): Muestra el resultado.

Lógica Requerida (Funciones)

Crea una función modular que devuelva el resultado convertido y una etiqueta de texto descriptiva mediante parámetros out o tuplas: